

```
[13]: import os
print(os.getcwd())
print(os.listdir())

/drive/notebooks/cind-820-learning-analytics/notebooks
['oulad_eda.ipynb', 'oulad_modelling.ipynb']
```

```
[14]: !mamba install pandas matplotlib numpy seaborn scikit-learn

mambajs 0.21.1

Specs: xeus-python, numpy, matplotlib, pillow, ipywidgets>=8.1.6, ipyleaflet, scipy, pandas, seaborn, scikit-learn
Channels: emscripten-forge-4x, conda-forge

Solving environment...
Solving took 0.8925 seconds
All requested packages already installed.
```

```
[15]: import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_auc_score

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

```
[10]: import os
print(os.getcwd())
print(os.listdir())

/drive/notebooks/cind-820-learning-analytics/notebooks
['oulad_eda.ipynb', 'oulad_modelling.ipynb']
```

```
[16]: project_root = ".."

processed_data_path = f"{project_root}/data/processed/"
fig_path = f"{project_root}/reports/figures/"

df = pd.read_csv(f"{processed_data_path}oulad_student_level.csv")

print(df.shape)
df.head()
```

(32593, 15)

```
[14]:
```

	code_module	code_presentation	id_student	gender	region	highest_education	imd_band	age_band	num_of_prev_attempts	studied_credits	disability
0	AAA	2013J	11391	M	East Anglian Region	HE Qualification	90-100%	55<=	0	240	N
1	AAA	2013J	28400	F	Scotland	HE Qualification	20-30%	35-55	0	60	N
2	AAA	2013J	30268	F	North Western Region	A Level or Equivalent	30-40%	35-55	0	60	Y
3	AAA	2013J	31604	F	South East Region	A Level or Equivalent	50-60%	35-55	0	60	N
4	AAA	2013J	32885	F	West Midlands Region	Lower Than A Level	50-60%	0-35	0	60	N

```
[18]: df["at_risk"] = df["final_result"].isin(["Fail", "Withdrawn"]).astype(int)

print(df["at_risk"].value_counts())
```

```
at_risk
1    17208
0    15385
Name: count, dtype: int64
```

```
[19]: feature_cols = [
    "code_module",
    "code_presentation",
    "gender",
    "region",
    "highest_education",
    "imd_band",
    "age_band",
    "disability",
    "num_of_prev_attempts",
    "studied_credits",
    "mean_score",
    "total_assessments",
    "mean_weight"
]

X = df[feature_cols]
y = df["at_risk"]
```

```
[20]: numeric_features = [
    "num_of_prev_attempts",
    "studied_credits",
    "mean_score",
    "total_assessments",
    "mean_weight"
]
```

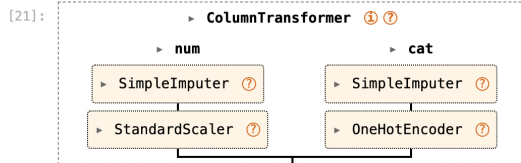
```
categorical_features = [
    "code_module",
    "code_presentation",
    "gender",
    "region",
    "highest_education",
    "imd_band",
    "age_band",
    "disability"
]
```

```
[23]: numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

preprocessor
```



```
[25]: X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print(X_train.shape, X_test.shape)
print(y_train.value_counts(normalize=True))
print(y_test.value_counts(normalize=True))

(26074, 13) (6519, 13)
at_risk
1    0.527959
0    0.472041
Name: proportion, dtype: float64
at_risk
1    0.527995
0    0.472005
Name: proportion, dtype: float64
```

```
[26]: # Logistic Regression
log_reg_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", LogisticRegression(max_iter=1000))
])

log_reg_pipeline.fit(X_train, y_train)
y_pred_log = log_reg_pipeline.predict(X_test)

print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_log))
print(classification_report(y_test, y_pred_log))

Logistic Regression Accuracy: 0.9038196042337782
precision    recall  f1-score   support

      0       0.88       0.93       0.90       3077
      1       0.93       0.88       0.91       3442

 accuracy          0.90          0.90          0.90       6519
 macro avg       0.90       0.91       0.90       6519
weighted avg       0.91       0.90       0.90       6519
```

```
[27]: tree_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", DecisionTreeClassifier(random_state=42, max_depth=5))
])

tree_pipeline.fit(X_train, y_train)
y_pred_tree = tree_pipeline.predict(X_test)

print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_tree))
print(classification_report(y_test, y_pred_tree))

Decision Tree Accuracy: 0.9292836324589661
precision    recall  f1-score   support

      0       0.88       0.98       0.93       3077
      1       0.98       0.89       0.93       3442

 accuracy          0.93          0.93          0.93       6519
 macro avg       0.93       0.93       0.93       6519
weighted avg       0.93       0.93       0.93       6519
```

```
[28]: rf_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", RandomForestClassifier(
        n_estimators=200,
        random_state=42,
        class_weight="balanced"
    ))
])
```

```

rf_pipeline.fit(X_train, y_train)
y_pred_rf = rf_pipeline.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))

```

	precision	recall	f1-score	support
0	0.90	0.97	0.93	3077
1	0.97	0.90	0.94	3442
accuracy			0.93	6519
macro avg	0.93	0.94	0.93	6519
weighted avg	0.94	0.93	0.93	6519

```

[29]: results = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree", "Random Forest"],
    "Accuracy": [
        accuracy_score(y_test, y_pred_log),
        accuracy_score(y_test, y_pred_tree),
        accuracy_score(y_test, y_pred_rf)
    ]
})

results = results.sort_values("Accuracy", ascending=False)
display(results)

results.to_csv(f"{processed_data_path}model_results.csv", index=False)

```

	Model	Accuracy
2	Random Forest	0.933732
1	Decision Tree	0.929284
0	Logistic Regression	0.903820

```

[30]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

rf_cv_scores = cross_val_score(
    rf_pipeline,
    X,
    y,
    cv=cv,
    scoring="accuracy"
)

print("Random Forest CV scores:", rf_cv_scores)
print("Mean CV accuracy:", rf_cv_scores.mean())

```

Random Forest CV scores: [0.93403896 0.93680012 0.93680012 0.94216017 0.93003989]
Mean CV accuracy: 0.9359678539672149

```

[31]: results.to_csv(f"{processed_data_path}model_results.csv", index=False)

```

```

[32]: cv_results = pd.DataFrame({
    "fold": [1, 2, 3, 4, 5],
    "rf_accuracy": rf_cv_scores
})

cv_results.to_csv(f"{processed_data_path}rf_cv_scores.csv", index=False)
cv_results

```

```

[30]:

```

	fold	rf_accuracy
0	1	0.934039
1	2	0.936800
2	3	0.936800
3	4	0.942160
4	5	0.930040

```

[ ]:

```